Git + GitHub for Two (or More) Machines — Elementary Version

For: You, moving from "Dropbox only" to "my code can live on my laptop **and** on Rivanna **and** still be one story."
You already know one piece of this: at West Point you used GitHub to **pull** new lesson plans. That was you saying: "GitHub has the newest version; copy it down to my computer." We will use the **same idea**, but you will also learn to **send** your work **up** to GitHub (`push`), and **download** it on another machine (`pull` or `clone`).

Related docs (same folder, `tenure_documents`):
`TENURE_STREAMLINING_AND_RESEARCH_PRIORITIES.md` — conda env names vs folder layout, how Git helps clutter, **research priorities** after connectivity, and **which .md files to print** for a reading packet.

Part 1 — Three boxes (picture this)

Think of three **boxes** that can hold the **same project**:

Box	Plain English	What it is
Box A	Your laptop	The folder on your Mac (still can live inside Dropbox if you want).
Box B	Rivanna (or any second computer)	Another place you work — same files, but a **different path** on the cluster.
Box C	GitHub	A **central** copy on the internet. Your **Git remote** (next section) points to GitHub.

Git is the tool that knows how to **compare** and **sync** these boxes **without** emailing zip files to yourself.

GitHub is a **website** that holds Box C and shows you history, buttons, and backups.

Part 2 — What is a "Git remote"? (The short answer)

A **remote** is a **nickname + address** for **Box C** — the GitHub copy.

- The usual nickname is `origin` (like "the main upstream place").
- The **address** looks like `https://github.com/username/repo-name.git` or the SSH form `git@github.com:username/repo-name.git`.

You are **not** doing anything magical. You are telling Git:

> *"When I say `origin`, I mean **that** GitHub repository."*

So:

- `git push` = "Send my new commits from this computer **to** `origin` (GitHub)."
- `git pull` = "Bring down whatever changed on `origin` since last time."

At West Point you only did `pull` (or the teacher told you to pull — same idea). **Now** you will also `push` (you are the teacher of your own repo).

Part 3 — How this connects to your old workflow

Old (lesson plans):

1. GitHub had the latest lesson.
2. You ran something like `git pull` (or clicked a button that did the same idea).
3. Your computer got the new files.

New (dissertation + Rivanna):

1. You **push** from your Mac when you finish a chunk of work → GitHub has the latest.
2. On Rivanna you `git pull` (or `git clone` the first time) → the cluster gets the same code.
3. You edit on Rivanna or run jobs, **commit**, **push** again if you changed code there.
4. Back on your Mac, `git pull` → you get those changes.

Rule of thumb: Before you start work on a machine, `git pull`. After you finish a logical chunk, `git commit` then `git push`. That keeps Box A, Box B, and Box C in agreement.

Part 4 — What goes in Git? (Not everything)

Git is **great for code**: `.py`, `.ipynb`, `.md`, small `.csv` configs.

Git is **bad for** huge data you already have on Rivanna (OpenAlex **terabytes**). **Do not** try to upload OpenAlex to GitHub.

Your setup: You have `cdh` → `Chas_Working` and OpenAlex folders. Those stay **on disk** where they are. In your code you **point** to paths (or environment variables). Git only stores the **scripts**, not the whole OpenAlex dump.

Part 5 — First-time setup (one time on your Mac)

These steps assume you log in to **GitHub** in the browser and create a **new repository**.

Step A — Create an empty repo on GitHub

1. Go to GitHub → **New repository**.
2. Name it something clear (e.g. `dissertation-pde` or `tenure-pipeline`).
3. Choose **Private** (still fine even if "nothing is private" — habit for later).

4. **Do not** add a README, `.gitignore`, or license **if** you already have a full folder locally (avoids a merge mess). If the repo is empty, GitHub will show you commands — we use those below.

Step B — Open Terminal on your Mac

`cd` to your project folder (the one you want to be versioned), e.g.:

```
cd "/Users/charleslevine/Library/CloudStorage/Dropbox/1-Documents/00- Dissertation/0-Next_Chapter/Code_and_Data/New SQL and PY
        Code/Cursor Workspace PDE"
```

(Use your real path if different.)

Step C — Turn this folder into a Git project

```
git init
```

That creates a hidden `.git` folder — Git's **memory** for this directory.

Step D — Add a `.gitignore` before the first commit (important)

This file tells Git: **ignore these files**. You want to ignore:

- Dropbox junk if any (`*.tmp`, sometimes `.DS_Store`)
- Python cache: `__pycache__/`, `*.pyc`
- Virtual environments: `venv/`, `.venv/`
- Secrets: `.env`, keys, passwords
- **Huge** datasets and outputs you do not want in GitHub

You can start small and grow. Example minimal `.gitignore`:

```
.DS_Store
__pycache__/
*.pyc
.env
.venv/
venv/
```

If your notebook outputs are huge, you can ignore specific outputs later — **not** your whole notebook.

Step E — Connect your folder to GitHub (`origin`)

GitHub shows you a URL. Use **HTTPS** (simpler first time) or **SSH** (if you already set up SSH keys with GitHub).

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
```

Replace with your real URL. To check:

```
git remote -v
```

You should see `origin` pointing at GitHub.

Step F — First save (commit) and first upload (push)

```
git status
git add .
git commit -m "Initial commit: dissertation pipeline workspace"
git branch -M main
git push -u origin main
```

GitHub may ask you to log in (browser or token). After this, **Box C** (GitHub) matches **Box A** (your Mac).

Part 6 — Second machine: Rivanna (first time)

Open SSH to Rivanna, then:

Option 1 — Clone (cleanest if you have nothing yet on Rivanna)

```
cd ~/wherever/you/want/code
git clone https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
cd YOUR_REPO_NAME
```

Now **Box B** has a full copy of the repo.

Option 2 — You already copied files by hand

Then `cd` into that folder, run `git init` only if it is **not** already a repo — **better**: delete the messy copy and **clone** once so you do not duplicate confusion.

Every day on Rivanna

```
cd /path/to/your/clone
git pull
# ... work ...
git add .
git commit -m "Describe what you changed"
git push
```

Part 7 — Simple “traffic rules” (so you do not fight yourself)

1. **Pull before you edit** when you switch machines (`git pull`).
2. **Commit often** small steps (one idea per message).
3. **Push** when you are done with a **chunk** so the other machine can see it.
4. If Git says **merge conflict** (both machines changed the same line), **stop** and ask for help or read a short conflict guide — **do not** panic; it is normal.

Part 8 — Your `cdh` / `Chas_Working` / OpenAlex (how it fits)

- `cdh` (Connected Data Hub) is **storage on Rivanna** (you linked it with `cd` or a symlink — good).
- `Chas_Working` is **your sandbox** for files you need.
- **OpenAlex** folders are **big data**. You **do not** put them in Git. You **do** put in Git a **small** file like `hpc_paths.example.env` or a line in `README`:

  `OPENALEX_ROOT=/path/to/your/openalex/on/rivanna`

  The real path can live in a **local file that is git-ignored** (`hpc_paths.env`) so you never commit secrets or wrong paths by mistake.

Part 8a — `talent_net`, `sports_net`, `tenure_net` (conda) vs "cloning environments"

Your **conda environments** are named things like `talent_net`, `sports_net`, `tenure_net`. Those go with **projects** (talent, sports, tenure) and folders like `tenure_pipeline/`, `tenure_documents/`.

- `git clone` brings **code** (and `environment.yml` / `requirements.txt` if you add them) to Rivanna — **not** a copy of your whole Mac conda folder.
- On the cluster you **recreate** an environment with `conda env create -f environment.yml` (or similar), not by cloning "the env" as if it were a repo.

Git helps keep **one story for source files** across machines. **Conda** still defines **what Python packages** each project uses. See the longer discussion in `TENURE_STREAMLINING_AND_RESEARCH_PRIORITIES.md`.

Part 9 — If you get stuck (common messages)

Message	Meaning
`not a git repository`	You are not inside a folder `git init` was run in. `cd` to the right place.
`failed to push` / `rejected`	Someone (or you on another machine) pushed first. Run `git pull`, fix conflicts if any, then `git push` again.
`Permission denied`	GitHub login: use HTTPS with a **personal access token** or set up SSH keys. GitHub's help pages walk through this.

Part 10 — Summary in one sentence

A Git remote is "the GitHub address of your project." You **push** from any machine to save there, **pull** on another machine to catch up — **same as lesson plans, but you are the author now.**

References

- GitHub: create a repo
- GitHub: HTTPS authentication (personal access tokens)
- Official Git book (free): https://git-scm.com/book/en/v2

Appendix — Research priorities (self-contained; same as `TENURE_STREAMLINING_AND_RESEARCH_PRIORITIES.md` Part 3)

After SSH / Git / Rivanna connectivity is working, the **first scientific priority** is **rough counts** to see if the research agenda is viable.

Question A — Certainty about role, place, and time: Count individuals with **high certainty** as **Assistant professor** at a **specific school** at a **specific time** (year / season / snapshot, per panel design).

Question B — Outcomes: Among those, quantify how many **promoted to Associate** (or equivalent) vs how many **left** the assistant rank without promotion (**"got out"** — define operationally: industry, other institution, etc.).

Question C — OpenAlex first: Restrict the **first** OA author disambiguation pass to **high-confidence** people who are **assistant professors**; **quantify N**. Those names feed OA matching; bulk OpenAlex on HPC follows for scale.

Question D — Performance on Rivanna: Use **downloaded OpenAlex** on the HPC for **works** and **citations** (and derived metrics) as performance measures for disambiguated authors.

Order of operations: (1) Panel / parse certainty → assistant + school + time counts. (2) High-confidence assistant subset → count + list. (3) OA disambiguation (API + HPC bulk as needed). (4) Works / citations → analysis.

You asked for elementary — this is the whole picture. When you create the GitHub repo and have the exact URL, you can paste the "push" / "clone" commands from GitHub's page and only replace names.